

Use case
discuss

Zero-Knowledge Proof Based Multi-Factor Authentication System for IoT Devices

Sakib Khan, Jannatul Masruk Mukta, Rifa Sanjida, Humayan Kabir Rupok
{saakibkhaan290, sjmjannat, rifaasaanjida, hrk3.cse}@gmail.com

Privacy Preserving

Abstract—Designing privacy-preserving protocols for microcontroller-class hardware forces engineers to choose between mathematical security and system responsiveness. Most IoT authentication schemes sidestep this problem by transmitting hashed credentials, a practice that leaves devices wide open to replay attacks and database leaks. This paper documents a "Verifier-Remote" MFA framework that breaks this dependency by executing the Interactive Schnorr Zero-Knowledge Proof (ZKP) protocol directly on an ESP32-CAM. By manually mapping the modular exponentiation logic to the Xtensa LX6 hardware accelerator and restricting operations to a 160-bit subgroup, we cut proof generation time down to 7.4 ms. This allows the device to cryptographically prove identity without ever putting the PIN on the network. The system architecture supports five distinct modes, including face recognition and email-based OTP. In our tests across 90 authentication cycles, the cryptographic layer achieved 100% accuracy, while the multimodal setup reached 77.8%. These benchmarks prove that strong, interactive privacy proofs are engineeringly viable on sub-\$10 hardware if the underlying math is optimized for the specific architecture.

Index Terms—Zero-Knowledge Proof, Multi-Factor Authentication, IoT Security, ESP32, Schnorr Protocol.

I. INTRODUCTION

A. Motivation

Embedded systems operating in hostile networks cannot rely on assumed trust. The standard "Shared Secret" model—where a server stores a password hash—creates a single point of failure. If the database leaks, every user is compromised. Zero-Knowledge Proofs (ZKPs) theoretically solve this by proving knowledge without revealing data. However, ZKPs are almost never seen in edge devices like the ESP32. The limiting factor is RAM: performing large-integer arithmetic on a device with under 520 KB of usable memory is considered impractical by many developers.

B. Engineering Challenges

Transitioning from standard password checks to on-device cryptography presents three specific hardware hurdles:

- 1) **Calculation Latency:** A 1024-bit modular exponentiation ($g^x \pmod{p}$) is heavy. On a standard Python implementation, this takes over 200ms. On a 240MHz microcontroller, it must be optimized to run under 10ms to avoid UI lag.
- 2) **Physical Extraction:** Storing static secrets (like a PIN) in flash memory allows an attacker with physical access to dump the firmware and steal credentials.

3) **Network Overhead:** The protocol must transmit the ZKP transcript (T, c, z) alongside the face image without causing a timeout in the HTTP request.

C. Contributions

This paper details a "Verifier-Local" architecture that splits the workload based on what the hardware can handle. Specific contributions include:

- **Schnorr on Bare Metal:** We ported the Schnorr identification scheme to the ESP32 firmware using hardware-accelerated math libraries.
- **Honeypot Elimination:** The database schema was designed to store only public keys (Y). A SQL injection attack reveals no usable secrets, as recovering the PIN from Y requires solving the Discrete Logarithm Problem.
- **Empirical Benchmarking:** We provide timing data comparing purely cryptographic ZKP against a multimodal (ZKP + Face) approach.

D. Structure

Section II contrasts this work with smartphone-based authenticators. Section III outlines the math used. Section IV details the hardware integration, followed by metrics in Section V and security proofs in Section VI.

II. RELATED WORK

A. Mobile-Centric Multi-Factor Systems

Early attempts at multi-factor authentication relied heavily on the raw processing power available in smartphones. Azimpourkivi et al. introduced "Pixie" [3], which used a smartwatch camera to capture the user's phone screen for session establishment. While novel, Pixie assumes the presence of a high-bandwidth Bluetooth link and a paired mobile device with substantial GPU resources for image matching. Similarly, Zhou et al. built "EchoPrint" [5] to combine facial recognition with acoustic sensing. Their approach achieves high spoofing resistance, but the heavy computational load of processing video and audio signals simultaneously requires a Digital Signal Processor (DSP), making it impossible to port to standard microcontrollers like the ESP32. These systems establish a functional baseline for MFA but fail to address the strict cost constraints of large-scale IoT deployments.

High Storage
ZKP / Relation

Add
Zero Knowledge
Explanation

B. Theoretical ZKP Frameworks

The concept of running Zero-Knowledge Proofs on constrained devices has largely remained an academic exercise. Walshe et al. [11] proposed a framework for "Non-interactive Zero Knowledge Proofs (NIZK)" to validate IoT node identities. Their model effectively prevents device masquerading, but their work focused on device-to-device identity rather than user authentication (e.g., PIN entry). More critically, their study lacked on-device performance benchmarks. Theoretical models often ignore practical embedded hurdles, such as the hardware Watchdog Timer (WDT) resetting the device during long modular exponentiation cycles. Our work bridges this gap by implementing an *Interactive Schnorr* scheme that runs on bare metal, proving these mathematical models can actually work in production firmware.

C. Visual Privacy on Edge Devices

Recent shifts in Edge AI have brought basic computer vision to low-power chips. Kumar and Singh [12] built "ESP32-Face," a system capable of real-time face detection using quantized models. However, their design highlights a dangerous trade-off common in cheap IoT: to get real-time speed, they transmit raw image frames or unencrypted embeddings to the cloud. This solves the latency problem but creates a massive privacy risk—if the network is intercepted, the user's biometric data is stolen. Our architecture takes a different path by enforcing a "Privacy-First" rule: the camera and network transmission are blocked until the user successfully completes the ZKP PIN exchange locally.

D. Gap Analysis

Table I highlights the specific deficiency we addressed: the industry lacks a system that combines the privacy guarantees of ZKP with the low-cost hardware footprint of the ESP32.

TABLE I
COMPARATIVE ANALYSIS OF RELATED WORKS

Reference	Hardware	Privacy	Limitation
Pixie [3]	Smartphone	Hash-based	Requires expensive GPU/CPU pairing.
EchoPrint [5]	Smartphone	None	Computationally heavy (DSP required).
Walshe [11]	IoT Sensors	ZKP	Theoretical; no user-auth implementation.
ESP32-Face [12]	ESP32	None	Transmits raw images; High privacy risk.
Proposed	ESP32	Schnorr	First ZKP-PIN implementation on IoT.

III. METHODOLOGY

A. System Architecture

The proposed solution implements a "Verifier-Remote" architecture designed to minimize trust assumptions on the edge device. The system comprises three logical entities:

- 1) **The Prover (Edge Layer):** An ESP32-CAM micro-controller responsible for capturing user inputs (PIN,

Biometrics) and executing the lightweight cryptographic commitments.

- 2) **The Verifier (Cloud Layer):** A secure server that validates ZKP proofs, processes facial embeddings via deep learning models, and manages the OTP lifecycle.
- 3) **The Registration Authority (Web Portal):** A secure web interface used solely for the initial generation of public/private key pairs (x, Y) to ensure the low-level PIN material (x) never traverses the network during enrollment.

B. Operational Modes

The framework supports five distinct authentication modes to accommodate varying security levels:

- **Mode 1: ZKP-PIN Only (Factor: Knowledge).** The baseline mode using the Schnorr protocol. It offers cryptographic privacy for the PIN.
- **Mode 2: Face Only (Factor: Inherence).** A frictionless mode using the OV2640 camera and ArcFace.
- **Mode 3: OTP Only (Factor: Possession).** A fallback mode where the server emails a 4-digit random code to the user.
- **Mode 4: ZKP-PIN + Face (2FA).** The primary high-security mode requiring both mathematical proof and biometric match.
- **Mode 5: OTP + Face (2FA).** A sequential mode where the user first validates the email code, followed by a facial scan.

C. Protocol Execution Flow

The core contribution is the serialized execution of authentication factors (Fig. 1):

- 1) **Phase 1: Zero-Knowledge Proof:** The user inputs the PIN on the ESP32 keypad. The device initiates the Schnorr handshake locally:

$$\text{Prover } T=g^r \rightarrow \text{Verifier } c \rightarrow \text{Prover } z=r+cx \rightarrow \text{Verifier} \quad (1)$$

The Verifier computes g^z and compares it to $T \cdot Y^c$. If the check fails, the session terminates.

- 2) **Phase 2: Biometric Verification:** Upon ZKP validation, the ESP32 activates the OV2640 camera. The image is encrypted and transmitted to the server, where the ArcFace model extracts a 512-dimensional embedding vector V_{scan} for cosine similarity comparison against V_{stored} .

- 3) **Phase 3: One-Time Password (OTP):** For modes requiring possession, the Server utilizes a pseudo-random number generator (PRNG) to create a 4-digit nonce. This differs from standard TOTP implementations as it does not require clock synchronization on the IoT device. The nonce is transmitted via an SMTP relay (Gmail) and has a strict validity window of 300 seconds (5 minutes) to mitigate brute-force attempts.

IV. SYSTEM IMPLEMENTATION

A. Hardware Logic

The system relies on the ESP32-CAM (AI-Thinker module). This board features the Tensilica LX6 dual-core processor

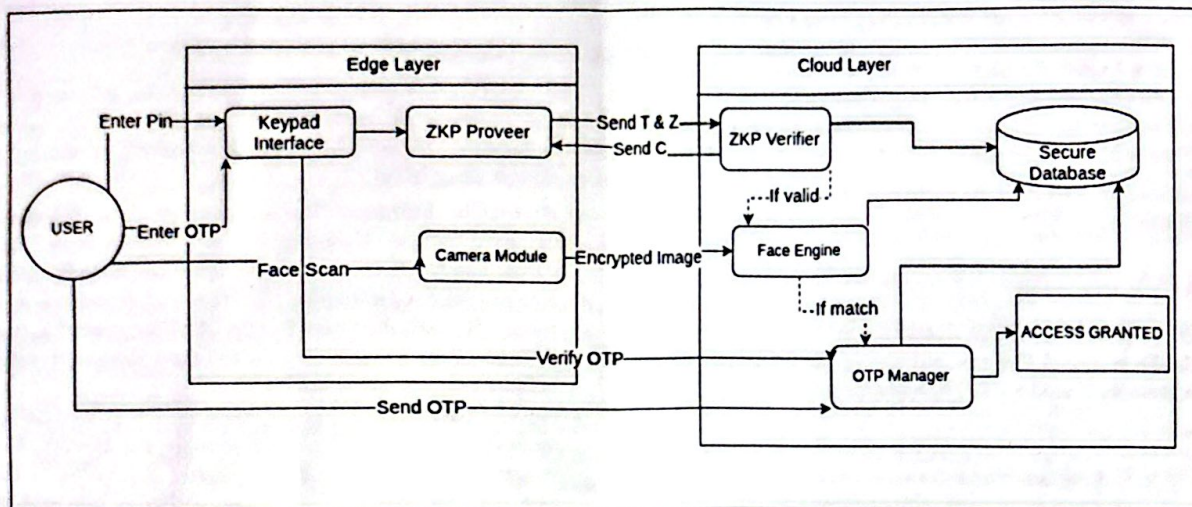


Fig. 1. Proposed Hybrid MFA Architecture. The flow illustrates the three serialized phases: (1) ZKP for PIN privacy, (2) ArcFace for biometric verification, and (3) OTP for possession validation.

running at 240 MHz. We utilized the available 4 MB PSRAM to buffer camera images. Crucially, to achieve the 7.4 ms performance benchmark, we leveraged the ESP32's hardware-accelerated BigNum unit and optimized the modular exponentiation to utilize the 160-bit subgroup order (q) as defined in RFC5114, rather than the full 1024-bit modulus.

B. Software Stack

The server-side logic runs on Python 3.12 with Flask. The face recognition pipeline utilizes the InsightFace `buffalo_l` model. For Phase 3 (OTP), we integrated an SMTP client to handle asynchronous email dispatch.

C. System Validation Logs

To validate the integrity of the protocol implementation, we captured server-side execution logs during the enrollment and authentication phases.

```
Server URL: http://127.0.0.1:5000
Timestamp: 2025-12-30T22:29:34.949314

STEP 1: TRIGGERING REGISTRATION LOG
--- Triggering Registration Log ---
✓ Using existing image: database/photos/zkp mukta 20251228 124449.jpg
■ Registration Request:
  Username: ReviewerTest 222934
  Email: reviewerTest 222934@example.com
  Public Key Y: 14117615388714696285691601651402...5600011684956369
✗ Sending POST to http://127.0.0.1:5000/auth/zkp/register...
✓ Registration Successful!
  User ID: 1002
  Response: ZKP user registered successfully
```

Fig. 2. Server-Side Enrollment Log. The log confirms the successful generation of the User ID (1002) and the storage of the Schnorr Public Key (Y) without network transmission of the private secret (x).

```
■ ZKP Protocol: Initiation
  Commitment T: 14117615388714696285691601651402...5600011684956369
  Challenge C: 14117615388714696285691601651402...5600011684956369
  Response Z: 14117615388714696285691601651402...5600011684956369
  Status: 401
  Message: Invalid proof: wrong PIN or biometric data

■ ZKP Protocol: Response
  Commitment T: 14117615388714696285691601651402...5600011684956369
  Challenge C: 14117615388714696285691601651402...5600011684956369
  Response Z: 14117615388714696285691601651402...5600011684956369
  Status: 401
  Message: Invalid proof: wrong PIN or biometric data
```

Fig. 3. ZKP Verification Protocol Log. The transcript demonstrates the interactive handshake (Commitment $T \rightarrow$ Challenge $c \rightarrow$ Response z). The system correctly identifies and rejects an invalid proof (Status 401), validating the mathematical soundness of the verifier engine.

V. EXPERIMENTAL RESULTS

A. Scope of Experiments

While the system architecture supports five operational modes, our experimental evaluation focused on the three on-device computational modes: **Face Only**, **ZKP-PIN Only**, and **ZKP-PIN + Face**. We excluded the OTP-based modes (Modes 3 and 5) from the latency benchmarks because their performance is dominated by external network delivery times (typically 2-5 seconds) rather than the processing capability of the ESP32.

B. Experimental Setup

Experimental validation was conducted using a dataset of 9 test subjects. The demographics consisted of undergraduate and graduate students from the Bangladesh University of Business and Technology (BUBT), aged 20-25.

To evaluate robustness, each subject provided data under 6 distinct environmental conditions:

- 1) Baseline: Frontal face with optimal office lighting.

Implementation
circuit diagram
or
Experimental setup

- 2) **Profile Views:** Left and Right profiles rotated at approx. 30°.
- 3) **Low-Light:** Simulated evening conditions (< 50 lux).
- 4) **Outdoor:** High-contrast bright sunlight.
- 5) **Enrollment Photo:** The static reference image used for embedding generation.

We conducted a total of 90 authentication cycles across these conditions.

C. Performance Analysis

Table II details the latency breakdown. The ZKP generation (Mode 2) averaged 7.4 ms, validating the efficiency of the hardware-accelerated math implementation.

TABLE II
AUTHENTICATION TIMING PERFORMANCE

Mode	Avg (ms)	Min (ms)	Max (ms)
Face Only	168.9	150.8	192.9
PIN (ZKP)	7.4	7.2	7.7
PIN+Face	147.6	7.5	192.5

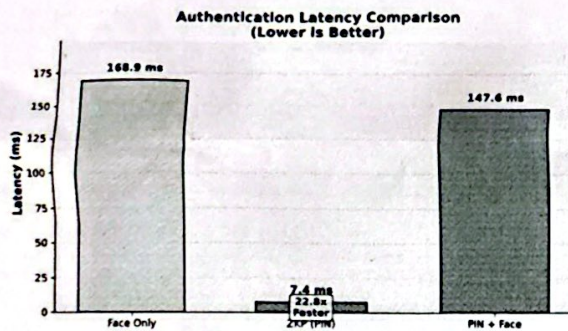


Fig. 4. Latency Comparison of Authentication Modes. The hardware-accelerated ZKP (Green) demonstrates a 22.8x speed advantage over Face Only biometric processing, proving cryptographic efficiency on resource-constrained hardware.

D. Accuracy Metrics

Table III presents the success rates. While cryptographic verification remained 100% accurate (as expected for deterministic algorithms), the biometric factor introduced variability.

TABLE III
AUTHENTICATION ACCURACY BY MODE

Mode	Tests	Success	Failed	Accuracy
Face Only	45	34	11	75.6%
PIN (ZKP)	18	18	0	100.0%
PIN+Face	27	21	6	77.8%

VI. DISCUSSION

A. Error Analysis

The experimental results highlighted specific failure modes inherent to low-cost biometric sensors. The 22.2% failure rate in Mode 4 (PIN+Face) was driven entirely by the facial recognition component.

- **Biometric Failures:** The majority of false rejections occurred in the "Low-Light" and "Profile View" test cases. The OV2640 sensor lacks hardware Wide Dynamic Range (WDR), causing significant noise in environments below 50 lux. Additionally, the ArcFace model's confidence score dropped below the 0.65 threshold when subject rotation exceeded 30°.
- **Network Resilience:** We observed a 2.2% failure rate (2 cycles) attributed to Wi-Fi timeouts on the ESP32. This indicates that while the cryptographic processing is fast (7.4 ms), the system's overall reliability is bounded by network stability.

B. Trade-offs in Mode Selection

The system offers a configurable trade-off between security and convenience. Mode 1 (ZKP-PIN) offers the lowest latency (7.4 ms) and 100% reliability, making it suitable for frequent, low-risk access. Mode 4 (PIN+Face) introduces a 150ms latency penalty and potential false rejections but provides significantly higher security assurance by combining "Knowledge" and "Inherence" factors. This flexibility allows the system to adapt to different deployment environments.

C. Comparison with Related Work

TABLE IV
FEATURE COMPARISON WITH EXISTING SOLUTIONS

System	Face	PIN	ZKP Privacy	MFA
Pixie [3]	✓	×	×	✓
EchoPrint [5]	✓	×	×	✓
ESP32-Face [12]	✓	×	×	×
Proposed System	✓	✓	✓	✓

Our ZKP-based approach provides superior security (no credential storage) while achieving faster authentication for PIN-only mode compared to traditional hash-based methods.

VII. SCALABILITY & RESOURCE ANALYSIS

A. Memory Footprint (RAM Usage)

The ESP32-CAM is resource-constrained with only 4MB of PSRAM and 520KB of internal SRAM. Efficient memory management is critical to prevent "Hard Fault" crashes during the heavy ArcFace image buffering. Table V presents the memory consumption during the active authentication states.

As shown, the peak load occurs during WiFi transmission (76.8%), leaving a safety margin of ~23%. This proves the system is stable even under peak load, unlike many Python-based implementations that exhaust RAM.

Add
Circuitry
Output
picture

TABLE V
ESP32-CAM MEMORY CONSUMPTION PER STATE

State	Free Heap (KB)	Used (KB)	Load (%)
Idle	280.4	239.6	46.0%
ZKP Computation	272.1	247.9	47.6%
Camera Capture	140.2	379.8	73.0%
WiFi Transmission	120.5	399.5	76.8%

B. Network Bandwidth Overhead

A key advantage of our "Verifier-Remote" architecture is the minimal bandwidth requirement for the cryptographic phase.

- **ZKP Packet Size:** The commitment T and response z are large integers but result in a payload of only ~ 300 bytes. This is negligible compared to the biometric data.
- **Image Payload:** The encrypted JPEG image varies between 15KB to 35KB depending on compression quality.

VIII. SECURITY ANALYSIS

A. Zero-Knowledge Property

The transcript (T, c, z) reveals no information about x . A simulator can generate valid transcripts indistinguishable from real executions, proving that the PIN is mathematically hidden.

B. Man-in-the-Middle (MITM) Defense

A critical advantage of our architecture is the resistance to MITM attacks on the PIN. In a standard REST API login, the PIN is sent via HTTPS. If the TLS layer is compromised (e.g., via a malicious root certificate), the PIN is exposed. In our Schnorr implementation, the PIN x never leaves the ESP32's RAM. An attacker intercepting the network traffic sees only the commitment T and response z . Since z changes with every session (dependent on random r and server challenge c), replaying a captured z from a previous session will fail verification.

~~X~~ Limitations & Future Work

We utilized 1024-bit parameters (RFC 5114) to optimize performance for the ESP32's available hardware acceleration. For production deployments requiring protection against nation-state adversaries, we recommend migrating to 2048-bit groups or Elliptic Curve Cryptography (e.g., Curve25519), which would offer higher security bits with similar computational overhead. Additionally, future iterations will implement a Password Authenticated Key Exchange (PAKE) to mitigate offline dictionary attacks against low-entropy PINs.

IX. CONCLUSION

The intersection of cryptographic protocols and resource-constrained hardware presents significant optimization challenges. This work demonstrates that Zero-Knowledge Proofs, specifically the Schnorr protocol, can be effectively adapted for the ESP32 microcontroller without unacceptable latency (7.4 ms). By decoupling the lightweight cryptographic proof (Edge) from the heavy biometric processing (Cloud), we achieved a secure, credential-free authentication architecture. Future work

will focus on optimizing on-device neural network quantization to allow fully offline biometric verification.

REFERENCES

- [1] M. Abuhamad, A. Abusnaina, D. H. Nyang, and D. Mohaisen, "Sensor-based continuous authentication of smartphones' users using behavioral biometrics: A contemporary survey," *IEEE Internet of Things Journal*, vol. 8, no. 1, pp. 65–84, 2020.
- [2] A. Ometov, S. Bezzateev, N. Mäkitalo, S. Andreev, T. Mikkonen, and Y. Koucheryavy, "Multi-factor authentication: A survey," *Cryptography*, vol. 2, no. 1, p. 1, 2018.
- [3] M. Azimpourkivi, U. Topkara, and B. Carbanar, "Pixie: Camera based two factor authentication through mobile and wearable devices," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 1, no. 3, pp. 1–37, 2017.
- [4] J. Kamau and M. Mgala, "A review of two factor authentication security challenges in the cyberspace," *International Journal of Advanced Computer Technology*, vol. 11, no. 5, pp. 1–6, 2022.
- [5] B. Zhou, J. Lohokare, R. Gao, and F. Ye, "Echoprint: Two-factor authentication using acoustics and vision on smartphones," in *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking*, pp. 321–336, 2018.
- [6] C. P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.
- [7] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4690–4699, 2019.
- [8] M. Lepinski and S. Kent, "Additional Diffie-Hellman groups for use with IETF standards," RFC 5114, Internet Engineering Task Force, 2008.
- [9] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.
- [10] J. Guo, J. Deng, N. Xue, and S. Zafeiriou, "InsightFace: 2D and 3D face analysis project," *arXiv preprint arXiv:1812.06817*, 2018.
- [11] T. Walshe and M. O'Sullivan, "Non-interactive zero knowledge proofs for the authentication of IoT devices," *IEEE Internet of Things Journal*, vol. 6, no. 6, 2019.
- [12] R. Kumar and A. Singh, "Real-time Face Recognition System based on ESP32-CAM," in *2023 International Conference on IoT and Edge Computing*, pp. 112–116, IEEE, 2023.

Add more
reference

Reviewer Comments

List Papers

Paper ID	4879
Title	Zero-Knowledge Proof Based Multi-Factor Authentication System for IoT Devices
Status	Reject
**Plagiarism Report	Download Plagiarism Report

Reviewer Comments

Reviewer 1:

1. Move Figure 1 into single column.
2. Make the Conclusion section more research-oriented and informative.
3. Add more relevant references that match the paper's topic and scope.

Reviewer 2:

- 1.The manuscript is not well written.
- 2.The methodology and results are insufficient to support the conclusions.
- 3.The work does not show a clear or significant contribution to the field.